

ICT600 CORRECTED STUDENT EDITION

February 2026 Corrected Student Scheme

Question-verified edition; no official answer scheme was supplied

Scope	February 2026 final examination
Evidence	Printed question, supplied scheme where available, current lecture notes, and technical verification
Status	100 marks: Part A 70, Part B 30

Prepared 26 September 2026

How to use this reference

Teaching-use document

Attempt the original question before revealing this scheme. This corrected edition is not an official UiTM replacement; it records known source errors and accepts equivalent technically correct answers.

- Assessment total: 100 marks: Part A 70, Part B 30.
- Model answers show one defensible solution; alternative correct wording or code may receive full credit.
- For code, method marks remain available when the approach is sound and a minor syntax error does not invalidate the solution.
- Security improvements are shown where historical examples trusted or displayed unvalidated input.

Q1

Four HTML functions and five web-based CMS tools.

[18 marks]

Model answer

- HTML functions: structure content, link/embed resources, build forms, and organise tabular/list content (2 each).
- Any five accepted course CMS/site-building tools with accurate identification (2 each).

Marking guide: Q1(a) 8; Q1(b) 10.

Q2

Website anatomy and client-side development tools.

[12 marks]

Model answer

- Containing block, logo, navigation, content, footer, whitespace (1 each).
- Any six valid development tools or clear category-example pairs (1 each).

Marking guide: 6 marks per subpart.

Q3(a)

Create a MySQL database using PHP.

[8 marks]

Model answer

- Connect (2), handle connection error (1), CREATE DATABASE (1), execute (1), success/database error handling (2), close (1).

Marking guide: Follow the full connection-execution-close sequence.

Q4

CodeIgniter Employee controller and MVC flow.

[10 marks]

Model answer

- CI3-compatible public index() loading employee_profile (4).
- Browser -> Employee controller/index -> load view -> Employee Profile -> rendered response to browser (6).

Marking guide: The supplied CI_Controller syntax indicates CodeIgniter 3.

Q5

Clear-text transmission and file-upload risks.

[12 marks]

Model answer

- Clear text: interception/tampering risk plus HTTPS/TLS and correctly configured certificates/no HTTP fallback (6).
- Upload: malicious file risk plus content/type validation, non-executable storage, and malware scanning (6).

Marking guide: Each subpart: risk 2 + two countermeasures 2 each.

Q3(b) - secure add.php exemplar

Study the complete working exemplar below.

[See printed allocation]

Model answer

- Equivalent correct implementations that satisfy the stated behaviour should receive credit.

```
<?php
$conn = mysqli_connect('localhost', 'app_user', 'app_password', 'myDB');
if (!$conn) die('Connection failed: ' . mysqli_connect_error());

$username = trim($_POST['username'] ?? '');
$password = $_POST['password'] ?? '';
$role = $_POST['role'] ?? 'user';
$status = $_POST['status'] ?? 'active';
$allowedRoles = ['user', 'staff'];
$allowedStatuses = ['active', 'inactive'];

if ($username === '' || $password === '' ||
    !in_array($role, $allowedRoles, true) ||
    !in_array($status, $allowedStatuses, true)) {
    die('Invalid input.');
```

```
}

$passwordHash = password_hash($password, PASSWORD_DEFAULT);
$stmt = mysqli_prepare($conn,
    'INSERT INTO users (username, password, role, status) VALUES (?, ?, ?, ?)');
mysqli_stmt_bind_param($stmt, 'ssss', $username, $passwordHash, $role, $status);
echo mysqli_stmt_execute($stmt) ? 'New record created successfully' : 'Insert failed';
mysqli_stmt_close($stmt);
mysqli_close($conn);
?>
```

Marking guide: Apply the original question's subpart allocation and award method marks consistently.

Correction / moderation note

The generated answer concatenates raw form values into SQL, stores the password in clear text, and accepts arbitrary role/status values. This corrected exemplar uses validation, allowlists, password hashing, and a prepared statement.

Part B Q1 - secure rental calculation core

Study the complete working exemplar below.

[See printed allocation]

Model answer

- Equivalent correct implementations that satisfy the stated behaviour should receive credit.

```
<?php
function e(string $v): string { return htmlspecialchars($v, ENT_QUOTES, 'UTF-8'); }
$name = trim($_POST['customerName'] ?? '');
$phone = trim($_POST['phone'] ?? '');
$package = $_POST['package'] ?? '';
$lateHours = filter_input(INPUT_POST, 'lateHours', FILTER_VALIDATE_INT,
    ['options' => ['min_range' => 0]]);
$rates = ['B' => 15, 'A' => 23, 'P' => 100];
if ($name === '' || $phone === '' || !isset($rates[$package]) || $lateHours === false) {
    die('Invalid input.');
```

```
}
$lateCharge = $package === 'P' ? 0 : $lateHours * 10;
$total = $rates[$package] + $lateCharge;
?>

<h1>Customer Details</h1>
<p>Name: <?= e($name) ?></p>
<p>Phone: <?= e($phone) ?></p>
```

```
<p>Package: <?= e($package) ?></p>
<p>Late return: <?= $lateHours ?> hour(s)</p>
<p>Charges: RM<?= number_format($total, 2) ?></p>
```

Marking guide: Apply the original question's subpart allocation and award method marks consistently.

Correction / moderation note

The generated calculation is plausible, but it trusts missing/invalid POST values and echoes raw input. This version validates and encodes output.

Part B Q2 - cumulative number analyser core

Study the complete working exemplar below.

[\[See printed allocation\]](#)

Model answer

- Equivalent correct implementations that satisfy the stated behaviour should receive credit.

```
const numbers = [];
function sendNumber() {
  const input = document.getElementById('number');
  const value = Number(input.value);
  if (input.value.trim() === '' || !Number.isFinite(value)) return;
  numbers.push(value);
  document.getElementById('values').textContent = numbers.join(', ');
  document.getElementById('maximum').textContent = Math.max(...numbers);
  document.getElementById('total').textContent =
    numbers.reduce((sum, number) => sum + number, 0);
  input.value = '';
  input.focus();
}
```

Marking guide: Apply the original question's subpart allocation and award method marks consistently.

Correction / moderation note

The generated logic is correct. textContent is used here because the outputs are text, not HTML.

Consistency rules

- Award equivalent technically correct syntax and wording.
- Do not reproduce a known historical error merely because it appears in a supplied scheme.
- Keep printed subpart totals fixed unless a documented moderation decision says otherwise.
- For scenario questions, connect the explanation or mitigation to the scenario.
- Record additional accepted alternatives for the next controlled revision.

February 2026 - Complete Coding Answers

Exam-schema presentation

Each answer below includes executable model code and an awardable marking schedule that sums to the printed marks. Equivalent working solutions receive the same marks when they satisfy the required behaviour.

Question 3(a) - create MySQL database

8 marks

Connect, create ict600_db, handle errors and close the connection.

Mark allocation

Awardable element	Marks
Connection	2
Connection error handling	1
CREATE DATABASE statement	1
Query execution	1
Success/query error output	2
Connection closed	1
Total	8

Complete model answer

```
<?php
$conn = mysqli_connect('localhost', 'root', '');
if (!$conn) {
    die('Connection failed: ' . mysqli_connect_error());
}

$sql = 'CREATE DATABASE ict600_db';
if (mysqli_query($conn, $sql)) {
    echo 'Database created successfully';
} else {
    echo 'Database creation failed: ' . mysqli_error($conn);
}
mysqli_close($conn);
?>
```

Question 3(b) - add.php

10 marks

Provide a complete User Entry form and safely insert a new user.

Mark allocation

Awardable element	Marks
Database connection	1
Retrieve and validate inputs	2
Password hash and role/status allowlists	2
Prepared INSERT and execution	3
Success/error output and close	1
Complete form and Back link	1
Total	10

Complete model answer

```
<?php
$message = '';
$conn = mysqli_connect('localhost', 'app_user', 'app_password', 'ict600_db');
if (!$conn) die('Connection failed: ' . mysqli_connect_error());

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
```

```

$username = trim($_POST['username'] ?? '');
$password = $_POST['password'] ?? '';
$role = $_POST['role'] ?? 'user';
$status = $_POST['status'] ?? 'active';
$roles = ['user', 'staff'];
$statuses = ['active', 'inactive'];

if ($username === '' || $password === '' ||
    !in_array($role, $roles, true) || !in_array($status, $statuses, true)) {
    $message = 'Invalid input.';
} else {
    $hash = password_hash($password, PASSWORD_DEFAULT);
    $stmt = mysqli_prepare($conn,
        'INSERT INTO users (username, password, role, status) VALUES (?, ?, ?, ?)');
    mysqli_stmt_bind_param($stmt, 'ssss', $username, $hash, $role, $status);
    $message = mysqli_stmt_execute($stmt)
        ? 'New record created successfully' : 'Unable to create record';
    mysqli_stmt_close($stmt);
}
}
mysqli_close($conn);
?>
<!doctype html><html lang="en"><body>
<h1>User Entry</h1>
<form method="post">
    <label>Username <input name="username" required></label><br>
    <label>Password <input type="password" name="password" required></label><br>
    <label>Role <select name="role"><option>user</option><option>staff</option></select></label><br>
    <label>Status <select name="status"><option>active</option><option>inactive</option></select></label><br>
    <button type="submit">OK</button>
</form>
<?php if ($message !== ''): ?>
<p><?= htmlspecialchars($message, ENT_QUOTES, 'UTF-8') ?></p>
<?php endif; ?>
<a href="index.php">Back</a>
</body></html>

```

Marker note

Prepared SQL and password hashing replace the insecure generated answer that concatenated raw input and stored a clear-text password.

Question 4(a) - CodeIgniter Employee controller

4 marks

Construct the CI3 index method that loads employee_profile.php.

Mark allocation

Awardable element	Marks
Employee extends CI_Controller	1
public index method	1
load->view call	1
Correct employee_profile view name/syntax	1
Total	4

Complete model answer

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

class Employee extends CI_Controller {
    public function index() {
        $this->load->view('employee_profile');
    }
}
?>

```

Part B Question 1(a) - inputData.html**5 marks****Create the complete bicycle-rental input form.****Mark allocation**

Awardable element	Marks
POST form/action	1
Name and phone	1
Three package choices	1
Late-hours numeric input	1
Submit and valid labels	1
Total	5

Complete model answer

```
<!doctype html>
<html lang="en"><body>
  <form action="calculate.php" method="post">
    <label>Name <input name="customerName" required></label><br>
    <label>Phone Number <input type="tel" name="phone" required></label><br>
    <fieldset><legend>Package</legend>
      <label><input type="radio" name="package" value="B" required> Basic</label>
      <label><input type="radio" name="package" value="A"> Advanced</label>
      <label><input type="radio" name="package" value="P"> Premium</label>
    </fieldset>
    <label>Late Return <input type="number" name="lateHours" min="0" value="0" required> hour/s</label><br>
    <button type="submit">Submit</button>
  </form>
</body></html>
```

Part B Question 1(b) - calculate.php**10 marks****Validate the submitted values, calculate base and late charges, and display the required customer table.****Mark allocation**

Awardable element	Marks
Retrieve/validate inputs	2
Package rates	2
Late-charge rule	2
Total calculation	1
Complete required output	2
Safe encoding/valid flow	1
Total	10

Complete model answer

```
<?php
function e(string $v): string { return htmlspecialchars($v, ENT_QUOTES, 'UTF-8'); }
$name = trim($_POST['customerName'] ?? '');
$phone = trim($_POST['phone'] ?? '');
$package = $_POST['package'] ?? '';
$late = filter_input(INPUT_POST, 'lateHours', FILTER_VALIDATE_INT,
  ['options' => ['min_range' => 0]]);
$rates = ['B' => 15, 'A' => 23, 'P' => 100];
if ($name === '' || $phone === '' || !isset($rates[$package]) || $late === false) {
  exit('Invalid input.');
```

```

<tr><td><? e($name) ?></td></tr>
<tr><td><? e($phone) ?></td></tr>
<tr><td>Package: <? e($package) ?></td></tr>
<tr><td><? $late ?> hour/s</td></tr>
<tr><td>Charges: <? number_format($total, 2) ?></td></tr>
</table>

```

Expected output / behaviour

For Ahmad, package B and 1 late hour: Charges: 25.00

Part B Question 2 - cumulative number analyser

15 marks

Provide the complete HTML and JavaScript page that stores every number and updates the list, maximum and cumulative total.

Mark allocation

Awardable element	Marks
Input, button and three output areas	3
Persistent array	2
Event and numeric validation	2
Append/display all values	2
Maximum	2
Cumulative total	2
Valid complete page and reset/focus behaviour	2
Total	15

Complete model answer

```

<!doctype html>
<html lang="en"><body>
<label>Number: <input id="number" type="number"></label>
<button type="button" id="send">Send</button>
<p>The value you entered: <span id="values"></span></p>
<p>The maximum number you entered: <span id="maximum"></span></p>
<p>The total number you entered: <span id="total"></span></p>
<script>
const numbers = [];
document.getElementById('send').addEventListener('click', () => {
  const input = document.getElementById('number');
  const value = Number(input.value);
  if (input.value.trim() === '' || !Number.isFinite(value)) return;
  numbers.push(value);
  document.getElementById('values').textContent = numbers.join(',');
  document.getElementById('maximum').textContent = Math.max(...numbers);

  document.getElementById('total').textContent =
    numbers.reduce((sum, number) => sum + number, 0);
  input.value = '';
});
</script>
</body></html>

```

Coding-marker consistency

- Award method marks when the core algorithm and required structure are correct.
- Do not double-penalise one syntax error that causes several later lines to fail.
- Accept equivalent DOM methods, loop forms, variable names, SQL aliases, and control structures.
- Require every behaviour named in the question; a prose description alone is not a complete coding answer.
- For unsafe historical code, award course-aligned logic while teaching prepared SQL, password hashing, validation, and output encoding.